Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of

Electrical & Computer Engineering

Lab Report 4

Implementation & Evaluation of Support Vector Machine (SVM)

Course Code: ECE 4246

Course Title: Machine Learning Sessional

Submitted to:
Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Submitted by:
Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Even

Submission Date: April 13, 2026

Contents

1 Theory and Introduction **1**
1.1 Support Vector Machine (SVM) 1
1.2 Gaussian Naive Bayes 2

2 Methodology **2**
2.1 Dataset 2
2.2 Algorithms 2
2.3 Code 3

3 Results and Discussion **8**
3.1 Support Vector Machine Results 8
3.2 Gaussian Naive Bayes Results 10

4 Conclusion **11**

References **11**

Implementation & Evaluation of Multi-Class Support Vector Machine (SVM) and Gaussian Naive Bayes

1 Theory and Introduction

This experiment explores two distinct approaches to multi-class classification: the Support Vector Machine (a discriminative, geometric maximum-margin classifier) and Gaussian Naive Bayes (a generative, probabilistic classifier). Both algorithms were implemented from scratch to evaluate their underlying mathematics and decision boundaries.

1.1 Support Vector Machine (SVM)

The Support Vector Machine (SVM) maps training examples to points in space to maximize the width of the gap between different categories [1]. Figure 1 illustrates the core components of this architecture, highlighting the optimal decision boundary, the maximized margin, and the critical support vectors that anchor the boundary.

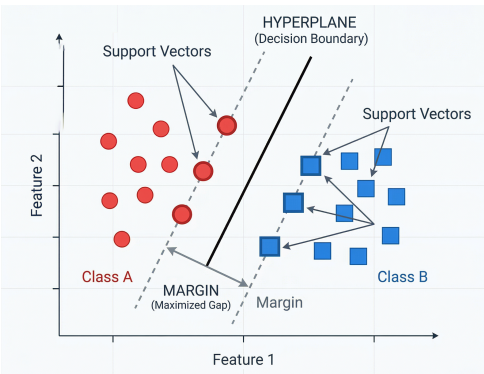


Figure 1: Conceptual illustration of a binary Support Vector Machine. The algorithm identifies the optimal hyperplane that maximizes the margin gap between Class A and Class B. The data points lying directly on the margin boundaries are the support vectors.

For a binary classification task where labels $y \in \{-1, 1\}$, a linear SVM minimizes the Hinge Loss with L_2 regularization:

$$J(w, b) = \lambda ||w||^2 + \frac{1}{n} \sum_{i=1}^{n} \max(0, 1 - y_i(w^T x_i + b)) \tag{1}$$

Standard SVMs are strictly binary classifiers. To classify K classes, K separate binary SVM models are trained using the One-vs-Rest (OvR) strategy. For a new input x, the class is assigned to the classifier yielding the highest confidence score:

$$\hat{y} = \arg \max_{k \in \{1,\ldots,K\}} (w_k^T x + b_k) \tag{2}$$

1.2 Gaussian Naive Bayes

Naive Bayes relies on Bayes' Theorem to predict the probability of a class y_k given an input feature vector X. It assumes that all features are mutually independent given the class [2].

$$P(y_k|X) = \frac{P(y_k) \prod_{i=1}^{n} P(x_i|y_k)}{P(X)} \tag{3}$$

For continuous data, it is assumed that the continuous values associated with each class are distributed according to a Gaussian (Normal) distribution. The likelihood $P(x_i|y_k)$ is calculated using the Gaussian Probability Density Function (PDF):

$$P(x_i|y_k) = \frac{1}{\sqrt{2\pi\sigma_{k,i}^2}} \exp\left(-\frac{(x_i - \mu_{k,i})^2}{2\sigma_{k,i}^2}\right) \tag{4}$$

Where $\mu_{k,i}$ and $\sigma_{k,i}^2$ are the mean and variance of feature i in class k. To avoid numerical underflow, the log-posterior is maximized during prediction instead of the raw probability product:

$$\hat{y} = \arg\max_k \left(\log P(y_k) + \sum_{i=1}^{n} \log P(x_i|y_k)\right) \tag{5}$$

2 Methodology

2.1 Dataset

The *Iris* dataset was utilized, containing three distinct classes (*Setosa*, *Versicolor*, and *Virginica*). The dataset was standardized. Two features (Petal Length and Petal Width) were used to allow for 2D visualization of the decision boundaries [3].

2.2 Algorithms

Algorithm 1 Multi-Class SVM (One-vs-Rest Training via SGD)

```
Input: Training Data (X, Y) with K classes, λ, α, epochs
Initialize: K instances of Binary SVMs
for k = 1 to K do
    Y_binary ← {1 if y_i == k else −1 for y_i ∈ Y}
    Train Model_k on (X, Y_binary) using Hinge Loss and SGD
end for
return array of trained models
```

Algorithm 2 Gaussian Naive Bayes (Training & Prediction)

function FIT(X, y)
  for each class $k \in K$ **do**
    $\mu_k \leftarrow$ Mean of X where $y = k$
    $\sigma_k^2 \leftarrow$ Variance of X where $y = k$
    $P(y_k) \leftarrow \frac{\text{Count}(y=k)}{\text{Total Samples}}$
  end for
end function
function PREDICT(x_{query})
  for each class $k \in K$ **do**
    $posterior_k \leftarrow \log P(y_k) + \sum \log \left(\text{GaussianPDF}(x_{query}, \mu_k, \sigma_k^2)\right)$
  end for
  return $\arg\max_k(posterior_k)$
end function

2.3 Code

The algorithms were implemented in Python using NumPy for matrix operations.

SVM Implementation

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# 1. EVALUATION & HELPER FUNCTIONS
def train_test_split_custom(X, y, test_size=0.2, seed=42):
    np.random.seed(seed)
    indices = np.random.permutation(len(X))
    split = int(len(X) * (1 - test_size))
    return (
        X[indices[:split]],
        X[indices[split:]],
        y[indices[:split]],
        y[indices[split:]],
    )


def standardize(X, mean_val=None, std_val=None):
    if mean_val is None or std_val is None:
        mean_val = np.mean(X, axis=0)
        std_val = np.std(X, axis=0)
        std_val[std_val == 0] = 1
    return (X - mean_val) / std_val, mean_val, std_val


def get_confusion_matrix_3class(y_true, y_pred):
    cm = np.zeros((3, 3), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1
    return cm


def evaluate_metrics_multiclass(y_true, y_pred):
    cm = get_confusion_matrix_3class(y_true, y_pred)
    accuracy = np.trace(cm) / np.sum(cm)

    precisions, recalls = [], []
    for i in range(3):
        tp = cm[i, i]
        fp = np.sum(cm[:, i]) - tp
        fn = np.sum(cm[i, :]) - tp

        precisions.append(tp / (tp + fp) if (tp + fp) > 0 else 0)
        recalls.append(tp / (tp + fn) if (tp + fn) > 0 else 0)

    macro_precision = np.mean(precisions)
    macro_recall = np.mean(recalls)
    macro_f1 = (
        (2 * macro_precision * macro_recall) / (macro_precision + macro_recall)
        if (macro_precision + macro_recall) > 0
        else 0
    )

    return accuracy, macro_precision, macro_recall, macro_f1, cm


# 2. PLOTTING FUNCTIONS
```

```python
def plot_multiclass_cost_history(models, class_names):
    plt.figure(figsize=(10, 5))
    colors = ["purple", "teal", "orange"]  # Distinct colors for the 3 curves

    for i, model in enumerate(models):
        plt.plot(
            model.cost_history,
            color=colors[i],
            linewidth=2,
            label=f"{class_names[i]} vs Rest",
        )

    plt.title("Multi-Class SVM Training: Hinge Loss vs. Epochs")
    plt.xlabel("Epochs")
    plt.ylabel("Cost (Hinge Loss)")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()


def plot_confusion_matrix(cm, class_names, title="Confusion Matrix"):
    fig, ax = plt.subplots(figsize=(6, 5))
    im = ax.imshow(cm, interpolation="nearest", cmap="Blues")
    ax.figure.colorbar(im, ax=ax)

    ax.set(
        xticks=np.arange(cm.shape[1]),
        yticks=np.arange(cm.shape[0]),
        xticklabels=class_names,
        yticklabels=class_names,
        title=title,
        ylabel="Actual Label",
        xlabel="Predicted Label",
    )

    thresh = cm.max() / 2.0
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(
                j,
                i,
                format(cm[i, j], "d"),
                ha="center",
                va="center",
                color="white" if cm[i, j] > thresh else "black",
                fontsize=14,
            )
    fig.tight_layout()
    plt.show()


def plot_multiclass_decision_boundary(X, y, model, class_names, title):
    plt.figure(figsize=(10, 6))
    colors = ["purple", "teal", "orange"]
    markers = ["o", "s", "^"]
    # Create mesh
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02), np.arange(y_min, y_max, 0.02))
    # Predict over mesh
    mesh_points = np.c_[xx.ravel(), yy.ravel()]
    Z = model.predict(mesh_points)
    Z = Z.reshape(xx.shape)
    # Plot filled contours for decision regions
    plt.contourf(xx, yy, Z, alpha=0.3, cmap="viridis")
    # Scatter actual points
    for i in range(3):
        plt.scatter(
            X[y == i, 0],
            X[y == i, 1],
            c=colors[i],
            marker=markers[i],
            edgecolors="k",
            s=60,
            label=class_names[i],
        )

    plt.title(title)
    plt.xlabel("Petal Length (Standardized)")
    plt.ylabel("Petal Width (Standardized)")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()


# 3. SVM ALGORITHMS (Base & OvR Wrapper)
class LinearSVM:
    def __init__(self, learning_rate=0.001, lambda_param=0.01, epochs=1000):
        self.lr = learning_rate
        self.lambda_param = lambda_param
        self.epochs = epochs
        self.w = None
        self.b = None
        self.cost_history = []  # ADDED: Initialize cost history array

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.w = np.zeros(n_features)
        self.b = 0
        self.cost_history = []  # Reset on fit

        for epoch in range(self.epochs):
            # ADDED: Calculate Hinge Loss for the epoch history
            distances = 1 - y * (np.dot(X, self.w) + self.b)
```

```python
            distances[distances < 0] = 0  # max(0, distance)
            hinge_loss = self.lambda_param * np.sum(self.w**2) + (
                1 / n_samples
            ) * np.sum(distances)
            self.cost_history.append(hinge_loss)

            for idx, x_i in enumerate(X):
                condition = y[idx] * (np.dot(x_i, self.w) + self.b) >= 1
                if condition:
                    dw = 2 * self.lambda_param * self.w
                    db = 0
                else:
                    dw = 2 * self.lambda_param * self.w - y[idx] * x_i
                    db = -y[idx]

                self.w -= self.lr * dw
                self.b -= self.lr * db

    def get_raw_score(self, X):
        return np.dot(X, self.w) + self.b


class MultiClassSVM:
    def __init__(self, n_classes=3, learning_rate=0.01, lambda_param=0.01, epochs=1000):
        self.n_classes = n_classes
        # Initialize an array of standard Binary SVMs
        self.models = [
            LinearSVM(learning_rate, lambda_param, epochs) for _ in range(n_classes)
        ]

    def fit(self, X, y):
        for i in range(self.n_classes):
            print(f"  -> Training SVM {i+1} (Class {i} vs Rest)...")
            # Convert target: 1 if class matches 'i', -1 otherwise
            y_binary = np.where(y == i, 1, -1)
            self.models[i].fit(X, y_binary)

    def predict(self, X):
        # Gather scores (w*x + b) from all 3 models
        scores = np.zeros((X.shape[0], self.n_classes))
        for i in range(self.n_classes):
            scores[:, i] = self.models[i].get_raw_score(X)

        # The predicted class is the one with the highest confidence score
        return np.argmax(scores, axis=1)


# 4. MAIN EXECUTION
def load_all_iris():
    url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
    try:
        df = pd.read_csv(url)
    except Exception as e:
        print(f"Error loading data: {e}")
        return None, None, None
    # Map all 3 classes to 0, 1, 2
    class_names = ["setosa", "versicolor", "virginica"]
    mapper = {name: i for i, name in enumerate(class_names)}
    df["species"] = df["species"].map(mapper)
    # 2 Features for Visualization
    X = df[["petal_length", "petal_width"]].values
    y = df["species"].values
    return X, y, class_names


def main():
    print("=" * 50)
    print(" MULTI-CLASS SVM (One-vs-Rest) CLASSIFICATION")
    print("=" * 50)
    # 1. Load Data
    X, y, class_names = load_all_iris()
    if X is None:
        return
    # 2. Split and Standardize
    X_train, X_test, y_train, y_test = train_test_split_custom(X, y, test_size=0.2)
    X_train_std, mean_val, std_val = standardize(X_train)
    X_test_std, _, _ = standardize(X_test, mean_val, std_val)
    # 3. Train Multi-Class SVM
    print("\nTraining Multi-Class SVM (One-vs-Rest strategy)...")
    ovr_svm = MultiClassSVM(
        n_classes=3, learning_rate=0.01, lambda_param=0.01, epochs=1500
    )
    ovr_svm.fit(X_train_std, y_train)
    # 4. Predict & Evaluate
    y_pred = ovr_svm.predict(X_test_std)
    acc, prec, rec, f1, cm = evaluate_metrics_multiclass(y_test, y_pred)

    print("\n" + "=" * 40)
    print("EVALUATION REPORT (3-Class)")
    print("=" * 40)
```

```python
    print(f"Accuracy:  {acc:.4f}")
    print(f"Precision (Macro): {prec:.4f}")
    print(f"Recall (Macro):    {rec:.4f}")
    print(f"F1 Score (Macro):  {f1:.4f}")
    print("\nConfusion Matrix:")
    print(cm)
    # 5. Visualizations. Plot all 3 Cost Histories (One for each SVM trained)
    plot_multiclass_cost_history(ovr_svm.models, class_names)

    plot_confusion_matrix(
        cm, class_names=class_names, title="Multi-Class SVM Confusion Matrix"
    )

    plot_multiclass_decision_boundary(
        X_test_std,
        y_test,
        ovr_svm,
        class_names=class_names,
        title="Multi-Class SVM Decision Regions",
    )


if __name__ == "__main__":
    main()
```

Naive Bayes Implementation

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# 1. EVALUATION & HELPER FUNCTIONS
def train_test_split_custom(X, y, test_size=0.2, seed=42):
    np.random.seed(seed)
    indices = np.random.permutation(len(X))
    split = int(len(X) * (1 - test_size))
    return (
        X[indices[:split]],
        X[indices[split:]],
        y[indices[:split]],
        y[indices[split:]],
    )


def standardize(X, mean_val=None, std_val=None):
    if mean_val is None or std_val is None:
        mean_val = np.mean(X, axis=0)
        std_val = np.std(X, axis=0)
        std_val[std_val == 0] = 1e-8
    return (X - mean_val) / std_val, mean_val, std_val


def get_confusion_matrix_3class(y_true, y_pred):
    cm = np.zeros((3, 3), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1
    return cm


def evaluate_metrics_multiclass(y_true, y_pred):
    cm = get_confusion_matrix_3class(y_true, y_pred)
    accuracy = np.trace(cm) / np.sum(cm)

    precisions, recalls = [], []
    for i in range(3):
        tp = cm[i, i]
        fp = np.sum(cm[:, i]) - tp
        fn = np.sum(cm[i, :]) - tp

        precisions.append(tp / (tp + fp) if (tp + fp) > 0 else 0)
        recalls.append(tp / (tp + fn) if (tp + fn) > 0 else 0)

    macro_precision = np.mean(precisions)
    macro_recall = np.mean(recalls)
    macro_f1 = (
        (2 * macro_precision * macro_recall) / (macro_precision + macro_recall)
        if (macro_precision + macro_recall) > 0
        else 0
    )

    return accuracy, macro_precision, macro_recall, macro_f1, cm


# 2. PLOTTING FUNCTIONS
def plot_confusion_matrix(cm, class_names, title="Confusion Matrix"):
    fig, ax = plt.subplots(figsize=(5, 4))
    im = ax.imshow(cm, interpolation="nearest", cmap="Blues")
    ax.figure.colorbar(im, ax=ax)

    ax.set(
        xticks=np.arange(cm.shape[1]),
        yticks=np.arange(cm.shape[0]),
        xticklabels=class_names,
        yticklabels=class_names,
        title=title,
        ylabel="Actual Label",
        xlabel="Predicted Label",
    )
```

```python

    thresh = cm.max() / 2.0
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(
                j,
                i,
                format(cm[i, j], "d"),
                ha="center",
                va="center",
                color="white" if cm[i, j] > thresh else "black",
                fontsize=14,
            )
    fig.tight_layout()
    plt.show()


def plot_decision_boundary(X, y, model, class_names, title):
    plt.figure(figsize=(9, 6))
    colors = ["purple", "teal", "yellow"]
    markers = ["o", "s", "^"]

    # Create meshgrid
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02), np.arange(y_min, y_max, 0.02))

    # Predict over mesh
    mesh_points = np.c_[xx.ravel(), yy.ravel()]
    Z = model.predict(mesh_points)
    Z = Z.reshape(xx.shape)

    # Plot contours
    plt.contourf(xx, yy, Z, alpha=0.3, cmap="viridis")

    # Scatter actual test points
    for i in range(3):
        plt.scatter(
            X[y == i, 0],
            X[y == i, 1],
            c=colors[i],
            marker=markers[i],
            edgecolors="k",
            s=60,
            label=class_names[i],
        )

    plt.title(title)
    plt.xlabel("Petal Length (Standardized)")
    plt.ylabel("Petal Width (Standardized)")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()


# 3. GAUSSIAN NAIVE BAYES ALGORITHM
class GaussianNaiveBayes:
    def __init__(self):
        self.classes = None
        self.mean = None
        self.var = None
        self.priors = None

    def fit(self, X, y):
        self.classes = np.unique(y)
        n_classes = len(self.classes)
        n_features = X.shape[1]

        # Initialize arrays for mean, variance, and prior probabilities
        self.mean = np.zeros((n_classes, n_features))
        self.var = np.zeros((n_classes, n_features))
        self.priors = np.zeros(n_classes)

        # Calculate statistics for each class
        for idx, c in enumerate(self.classes):
            X_c = X[y == c]
            self.mean[idx, :] = X_c.mean(axis=0)
            self.var[idx, :] = X_c.var(axis=0)
            self.priors[idx] = X_c.shape[0] / float(X.shape[0])

    def _calculate_likelihood(self, class_idx, x):
        """Gaussian Probability Density Function (PDF)"""
        mean = self.mean[class_idx]
        var = self.var[class_idx]
        # Add small epsilon to variance to prevent division by zero
        numerator = np.exp(-((x - mean) ** 2) / (2 * var + 1e-8))
        denominator = np.sqrt(2 * np.pi * var + 1e-8)
        return numerator / denominator

    def predict(self, X):
        y_pred = [self._predict_sample(x) for x in X]
        return np.array(y_pred)

    def _predict_sample(self, x):
        posteriors = []

        # Calculate posterior probability for each class
        for idx, c in enumerate(self.classes):
            prior = np.log(self.priors[idx])
            # Sum of logs is used instead of multiplying probabilities to prevent underflow
```

```python
            posterior = np.sum(np.log(self._calculate_likelihood(idx, x) + 1e-8))
            posterior = prior + posterior
            posteriors.append(posterior)

        # Return class with highest posterior probability
        return self.classes[np.argmax(posteriors)]


# 4. DATA LOADER & MAIN EXECUTION
def load_all_iris():
    url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
    try:
        df = pd.read_csv(url)
    except Exception as e:
        print(f"Error loading data: {e}")
        return None, None, None

    class_names = ["setosa", "versicolor", "virginica"]
    mapper = {name: i for i, name in enumerate(class_names)}
    df["species"] = df["species"].map(mapper)

    # 2 Features for Visualization
    X = df[["petal_length", "petal_width"]].values
    y = df["species"].values
    return X, y, class_names


def main():
    print("=" * 50)
    print(" GAUSSIAN NAIVE BAYES CLASSIFICATION")
    print("=" * 50)

    # 1. Load Data
    X, y, class_names = load_all_iris()
    if X is None:
        return

    # 2. Split and Standardize
    X_train, X_test, y_train, y_test = train_test_split_custom(X, y, test_size=0.2)
    X_train_std, mean_val, std_val = standardize(X_train)
    X_test_std, _, _ = standardize(X_test, mean_val, std_val)

    # 3. Train Naive Bayes
    print("\nTraining Gaussian Naive Bayes Model...")
    nb_model = GaussianNaiveBayes()
    nb_model.fit(X_train_std, y_train)

    # 4. Predict & Evaluate
    y_pred = nb_model.predict(X_test_std)
    acc, prec, rec, f1, cm = evaluate_metrics_multiclass(y_test, y_pred)

    print("\n" + "=" * 40)
    print("EVALUATION REPORT (3-Class)")
    print("=" * 40)
    print(f"Accuracy:  {acc:.4f}")
    print(f"Precision (Macro): {prec:.4f}")
    print(f"Recall (Macro):    {rec:.4f}")
    print(f"F1 Score (Macro):  {f1:.4f}")
    print("\nConfusion Matrix:")
    print(cm)

    # 5. Visualizations
    plot_confusion_matrix(
        cm, class_names=class_names, title="Naive Bayes Confusion Matrix"
    )

    plot_decision_boundary(
        X_test_std,
        y_test,
        nb_model,
        class_names=class_names,
        title="Gaussian Naive Bayes Decision Regions",
    )


if __name__ == "__main__":
    main()
```

3 Results and Discussion

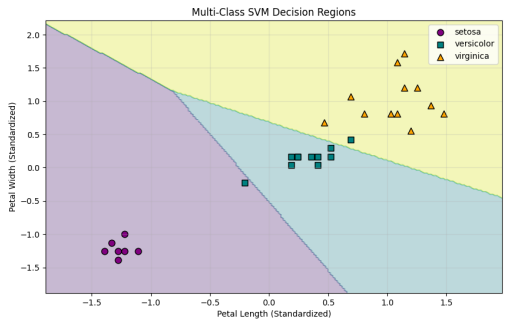
3.1 Support Vector Machine Results

The Multi-Class SVM was trained using $K = 3$ binary classifiers with a learning rate of 0.01, a lambda parameter of 0.01, over 1500 epochs.

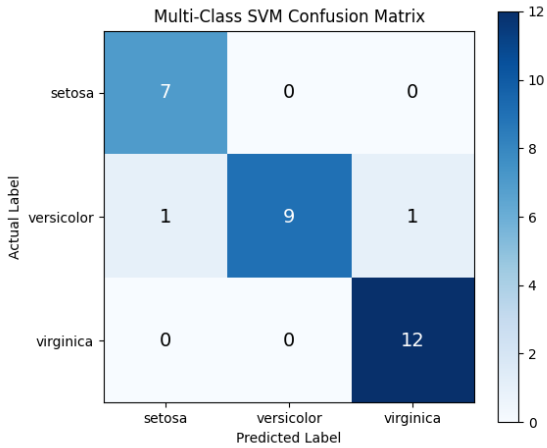
```
==================================================
 MULTI-CLASS SVM (One-vs-Rest) CLASSIFICATION
```

```
==================================================
Loading Iris Dataset...
Training Multi-Class SVM (One-vs-Rest strategy)...
  -> Training SVM 1 (Class 0 vs Rest)...
  -> Training SVM 2 (Class 1 vs Rest)...
  -> Training SVM 3 (Class 2 vs Rest)...

========================================
EVALUATION REPORT (3-Class)
========================================
Accuracy:  0.9333
Precision (Macro): 0.9327
Recall (Macro):    0.9393
F1 Score (Macro):  0.9350

Confusion Matrix:
[[ 7  0  0]
 [ 1  9  1]
 [ 0  0 12]]
```

(a) Hinge Loss vs. Epochs



(b) Decision Regions (OvR Boundary)



(c) Multi-Class SVM Confusion Matrix

Figure 2: Multi-Class SVM Performance Overview

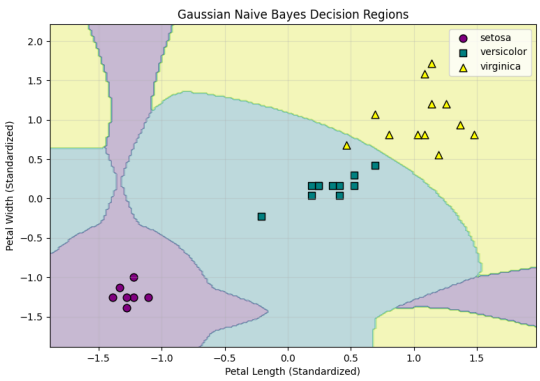
The SVM achieved an accuracy of 93.33%. The cost function (Figure 2a) demonstrates distinct convergence behavior. Separating the middle cluster (*Versicolor*) from the extremes yields a higher

residual cost compared to the other two classes. The decision boundary (Figure 2b) reveals the sharp, piecewise linear angles characteristic of the One-vs-Rest geometric approach.

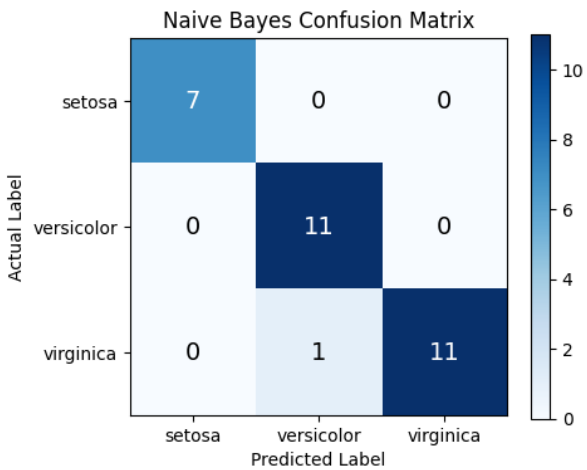
3.2 Gaussian Naive Bayes Results

The Gaussian Naive Bayes model was trained by analytically calculating the means and variances of the dataset, requiring zero iterative epochs.

```
==================================================
 GAUSSIAN NAIVE BAYES CLASSIFICATION
==================================================

Training Gaussian Naive Bayes Model...

========================================
EVALUATION REPORT (3-Class)
========================================
Accuracy:  0.9667
Precision (Macro): 0.9722
Recall (Macro):    0.9722
F1 Score (Macro):  0.9722

Confusion Matrix:
[[ 7  0  0]
 [ 0 11  0]
 [ 0  1 11]]
```



(a) Gaussian Naive Bayes Decision Regions



(b) Naive Bayes Confusion Matrix

Figure 3: Gaussian Naive Bayes Performance Overview

The Naive Bayes model outperformed the SVM on this dataset slice, achieving 96.67% accuracy. The confusion matrix (Figure 3b) shows only a single misclassification: one *Virginica* sample predicted as *Versicolor*.

In contrast to the strict linear cuts of the SVM, the decision boundary of Naive Bayes (Figure 3a) features curved (quadratic) lines. Because the covariance matrices between the features are not assumed to be equal, the intersection of their Gaussian probability distributions results in naturally curved boundaries, which better capture the overlapping spread of the *Versicolor* and *Virginica* clusters.

4 Conclusion

This experiment successfully demonstrated the contrast between discriminative (SVM) and generative (Naive Bayes) multi-class classification.

- **Support Vector Machine** accurately classified the data (93.33%) by solving an optimization problem to find maximum-margin hyperplanes. The One-vs-Rest strategy successfully merged these binary models into a rigid, piecewise linear decision space.
- **Gaussian Naive Bayes** achieved superior accuracy (96.67%) without the need for iterative gradient descent. By modeling the underlying probability distributions of the features, it constructed fluid, quadratic decision boundaries that effectively mapped the overlapping variance of the natural data.

References

[1] C. M. Bishop, *Pattern recognition and machine learning.* springer, 2006.

[2] T. Hastie, R. Tibshirani, and J. Friedman, *The elements of statistical learning: data, mining, inference, and prediction.* Springer Science & Business Media, 2009.

[3] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of eugenics*, vol. 7, no. 2, pp. 179–188, 1936.